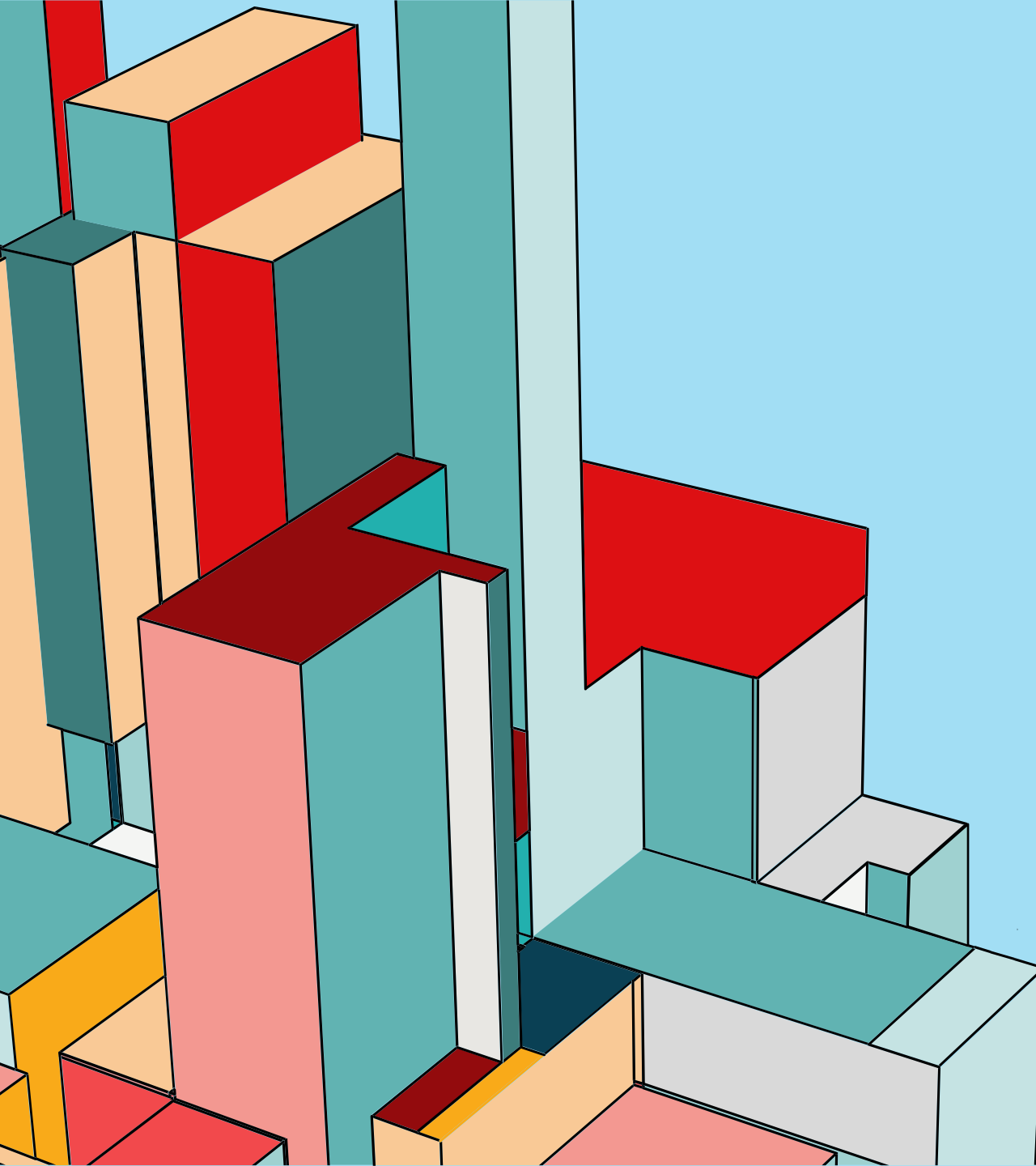


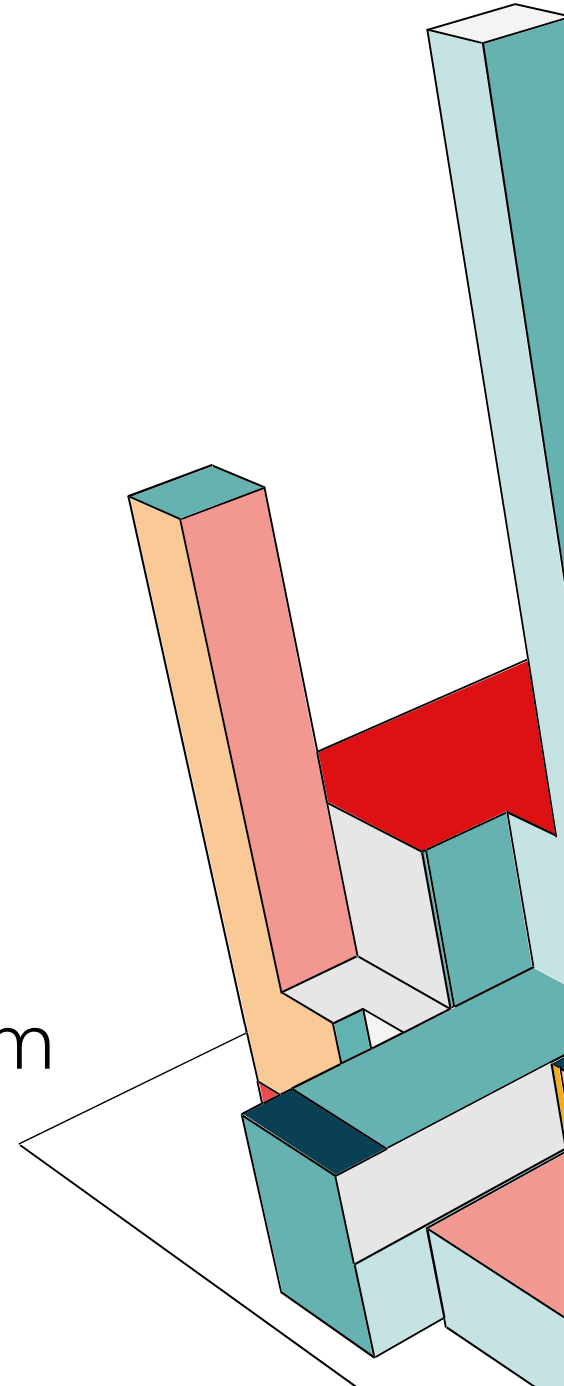


LECTURE 13 **HASHING**

CSIS 2300 Introduction to
Algorithms, Spring 2026
Dr. Dan DiTursi

ANNOUNCEMENTS & REMINDERS

- HW 6 due Tuesday night at 9pm
- Exam 2 next week on Wednesday (3/18)
- Fall registration opens in two weeks
- TA Chris's dissertation defense TODAY at 10am in Sage 5101



TRADING SPACE FOR TIME

A useful question to consider when designing an algorithm is this: Can I possibly save time if I use more space?



As an obvious example, a doubly-linked list uses more storage than a single-linked list. So why use one?

Because some tasks are faster if you can easily move both forwards and backwards through the list.

3 Another easy example is *counting sort*, but first...

SUBMITTY POLL #1

What's your favorite Tim Curry role?



COUNTING SORT

Consider this simple idea: If an element e has k smaller items in a list, then when the list is sorted, it should go to index k (assuming 0-based indexing).

So, to sort a list, we just perform this count for each element.

But how do we get **these counts**?

Naive idea: Check every pair of elements; increment the counter for the larger one.

This, of course, is $\Theta(n^2)$. Why bother?

i	A	count lesser	sort
0	24	7	3
1	15	4	6
2	6	1	7
3	38	10	13
4	17	5	15
5	3	0	17
6	19	6	19
7	40	11	24
8	7	2	26
9	26	8	31
10	31	9	38
11	13	3	40

COUNTING SORT

What if we are assured that every element in the array is $< k$ for some smallish integer k . Does that help?

(Let's use $k = 12$ in our example.)

Cool idea: Let's just count how many copies of each actual value we have.

How can we do that?

This one's pretty easy: we just use each value in A as an index in the count array and increment the value at that index.

i	A	count ==	Literally:
0	5	1	one 0
1	1	1	one 1
2	4	0	no 2s
3	5	0	no 3s
4	10	2	two 4s
5	8	3	three 5s
6	0	0	no 6s
7	4	0	no 7s
8	9	1	one 8
9	11	2	two 9s
10	5	1	one 10
11	9	1	one 11

This can be done in a single pass: $\Theta(n)$

COUNTING SORT – THE COUNT

We initialize all counts to 0, and then just walk down the list and update the appropriate entry in the count array.

```
for i = 0 to k:  
    count[A[i]]++
```

<i>i</i>	A	count ==
0	5	0
1	1	1
2	4	0
3	5	0
4	10	1
5	8	2
6	0	0
7	4	0
8	9	0
9	11	0
10	5	1
11	9	0

COUNTING SORT

Once we have the counts, now what?

If all we were doing was producing a list of numbers, we can just process the count array from 0 to 11 and fill in the appropriate number of copies of each element.

Usually, though, we're not just sorting numbers – the values that we're sorting are keys or identifiers that are attached to other data. So we need to deal with the specific elements of A.

<i>i</i>	A	count ==	sort
0	5	1	0
1	1	1	1
2	4	0	4
3	5	0	4
4	10	2	5
5	8	3	5
6	0	0	5
7	4	0	8
8	9	1	9
9	11	2	9
10	5	1	10
11	9	1	11

COUNTING SORT

We can use a running sum of counts to make an array listing how many elements in our list are less than or equal to a given number, which is very similar to what we had before.

Each of these values is one more than the index at which we want to store the last copy of each number.

So we can just subtract one from the value in the running total array before placement.

To make the sort stable, when placing the items of A into the output array, we will go in reverse order, starting at the last index of A.

<i>i</i>	A	count ==	count <=	Last __ goes at __
0	5	1	1	0 ... 0 (1-1)
1	1	1	2	1 ... 1
2	4	0	2	2 ... 1*
3	5	0	2	3 ... 1*
4	10	2	4	4 ... 3
5	8	3	7	5 ... 6
6	0	0	7	6 ... 6*
7	4	0	7	7 ... 6*
8	9	1	8	8 ... 7
9	11	2	10	9 ... 9
10	5	1	11	10 ... 10
11	9	1	12	11 ... 11

* None of these in A

COUNTING SORT

```
for i = k-1 down to 0:  
    val = A[i]  
    C[val] = C[val] - 1  
    copy A[i] to Z[C[val]]
```

Counting sort visualization:

<https://www.cs.usfca.edu/~galles/visualization/CountingSort.html>

<i>i</i>	A	count ==	count <=	sort
0	5	1	1	
1	1	1	2	
2	4	0	2	
3	5	0	2	4
4	10	2	3	
5	8	3	6	
6	0	0	7	5
7	4	0	7	
8	9	1	8	9
9	11	2	8	9
10	5	1	11	
11	9	1	11	11

FULL COUNTING SORT PSEUDOCODE

```
# A = input array of n values in range [0,k)
# Z = output array
def CountingSort(A, Z, k, n):
    initialize a size-k array C with all zeroes
    for i = 0 to n-1:  # initial counts
        C[A[i]] += 1
    for j = 1 to k-1:  # running sums
        C[j] += C[j-1]
    for i = n-1 down to 0:  # placements
        C[A[i]] -= 1
        Z[C[A[i]]] = A[i]
```

COUNTING SORT ANALYSIS

```
def CountingSort(A, Z, k, n):  
    initialize a size-k array C with all zeroes  $\Theta(k)$   
    for i = 0 to n-1:  $\Theta(n)$   
        C[A[i]] += 1  
    for j = 1 to k-1:  $\Theta(k)$   
        C[j] += C[j-1]  
    for i = n-1 down to 0:  $\Theta(n)$   
        C[A[i]] -= 1  
        Z[C[A[i]]] = A[i]
```

So the whole thing is $\Theta(n + k)$; as long as $k = O(n)$, this is $\Theta(n)$.

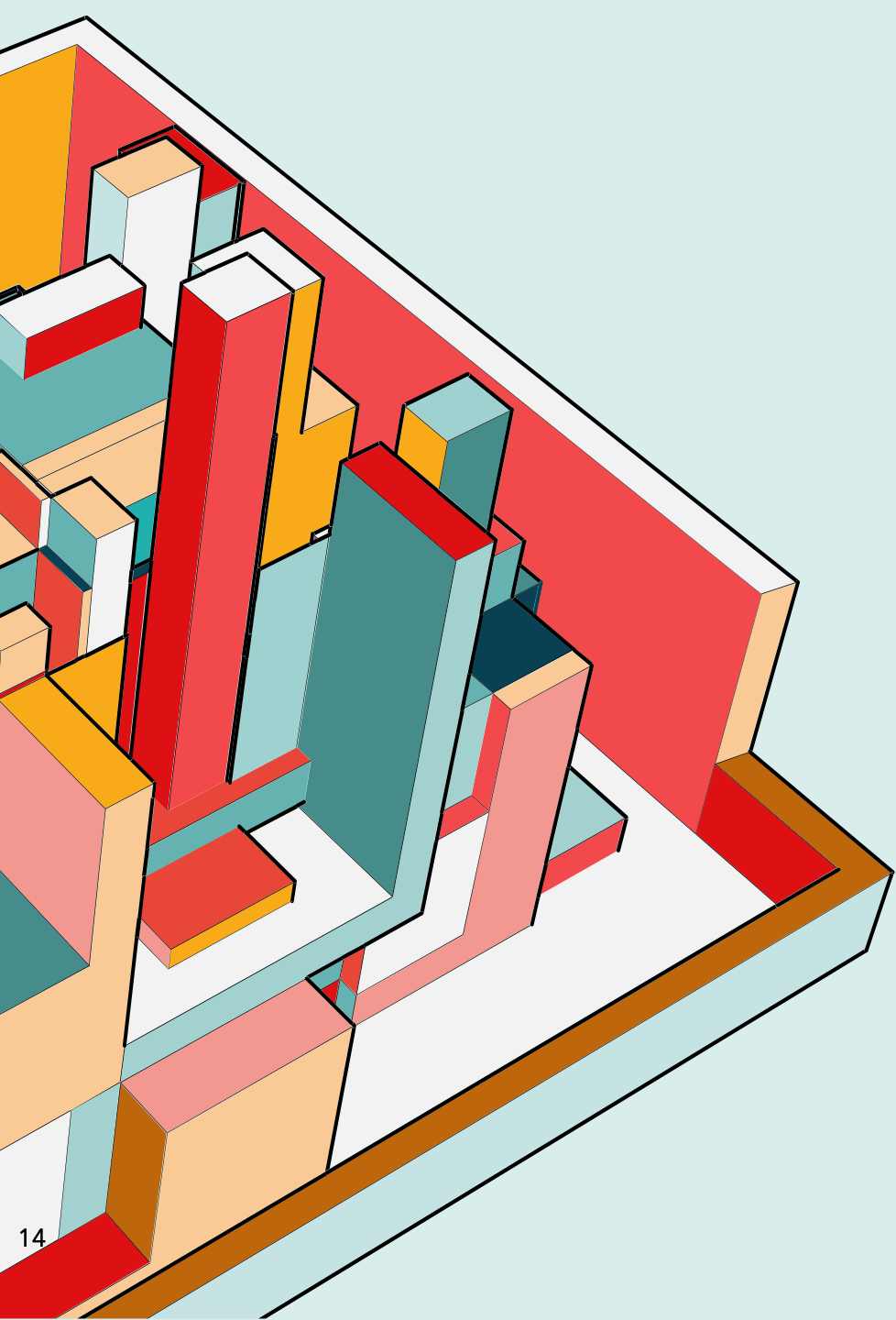
COUNTING SORT OBSERVATIONS

Notice that, like radix sort, counting sort avoids the $O(n \log n)$ bound by not comparing elements – at no point in the process do we ever check if $A[i] < A[j]$.

Another similarity is that counting sort is not in-place: we need a fair bit of extra space. Specifically, we need the arrays C and Z , which is $\Theta(n + k)$ additional memory.

In fact, radix sort is generally implemented using counting sort:

```
def RadixSort(A,d): # keys have d digits, where
    for i = 0 to d-1: # digit 0 is least significant
        Run counting sort on A using digit i as the key.
```



HASHING

DICTIONARY OPERATIONS

There are many situations in which we need to maintain a *dictionary*: a dynamic set of objects, each of which has a key or identifier. Dictionaries require, at a minimum, three operations:

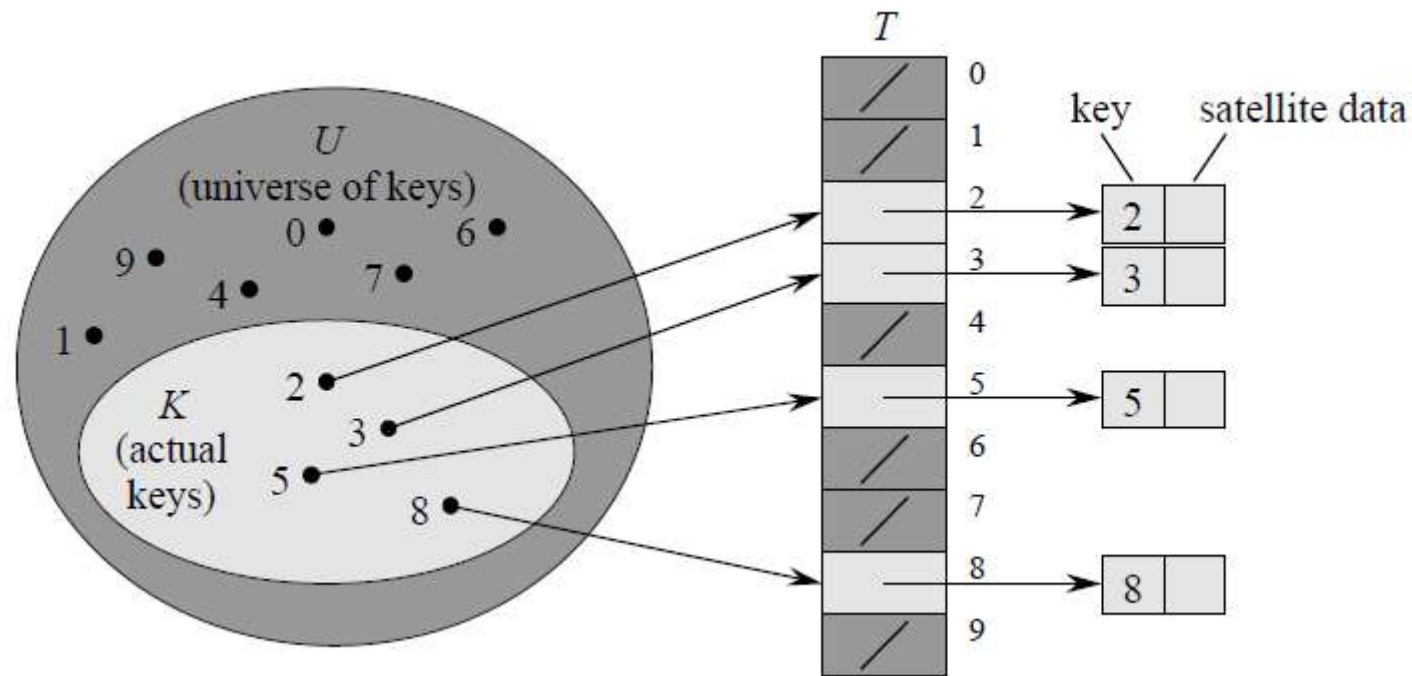
- Insert an element
- Find an element (by its key)
- Delete an element

There are, of course, lots of possible data structures we could use for this, but the core objective is to do all of these quickly.

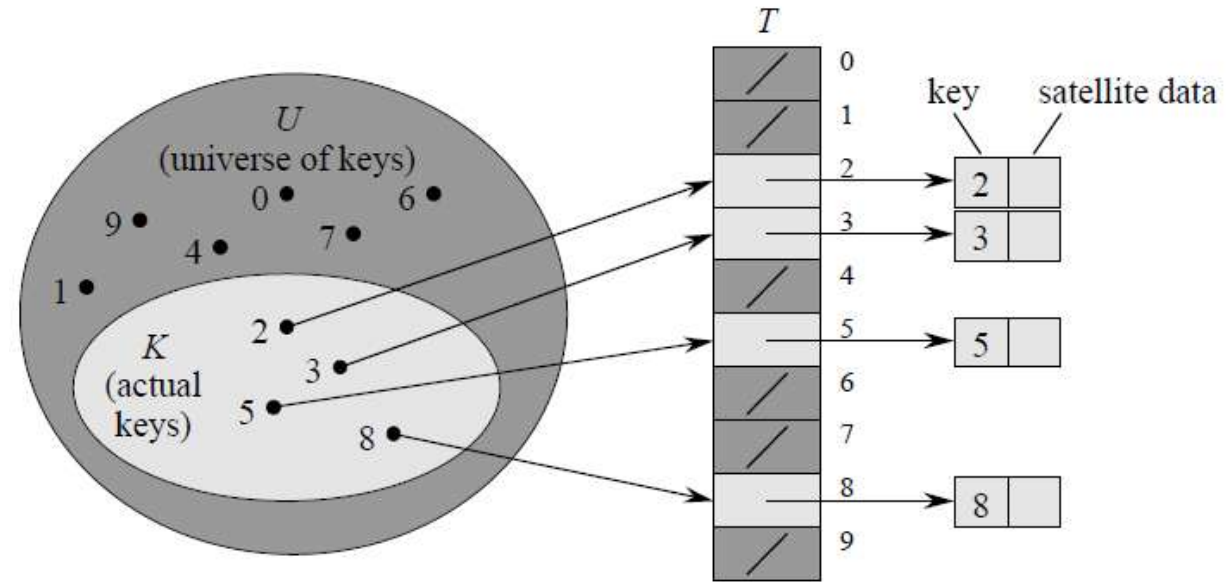
DIRECT-ADDRESS TABLE

The count array created by counting sort is a form of *direct-address table*. In such a table, the object key/ID is the literal index at which we will store that object.

Want to store something with ID 17? It goes into **A[17]**.



DIRECT-ADDRESS TABLE



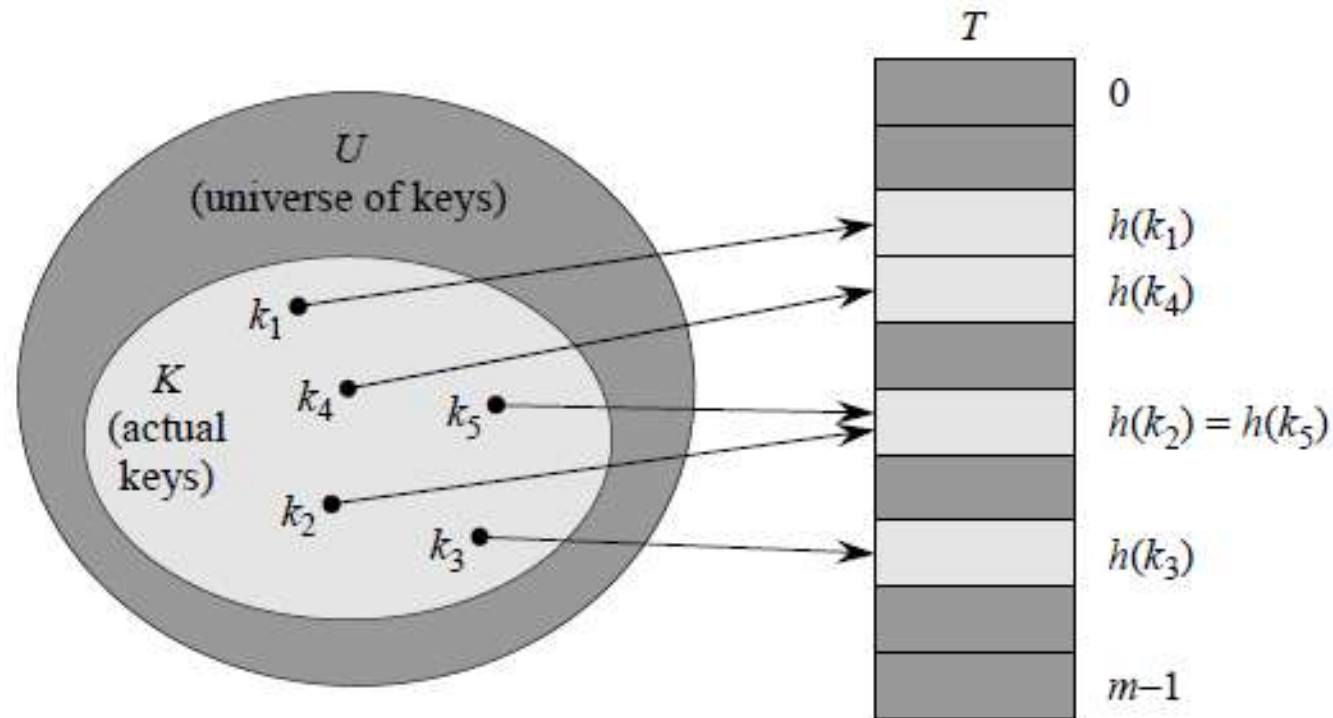
When keys are unique, this is the ultimate expression of the space-for-time tradeoff. How long does it take to insert an element? Find an element? Delete an element?

Each of these is a constant-time operation!

To accommodate this speed, we need to allocate space for $|U|$ elements, where $|U| - 1$ is the maximum possible key. In many applications $|U| \gg n$, which makes this prohibitive.

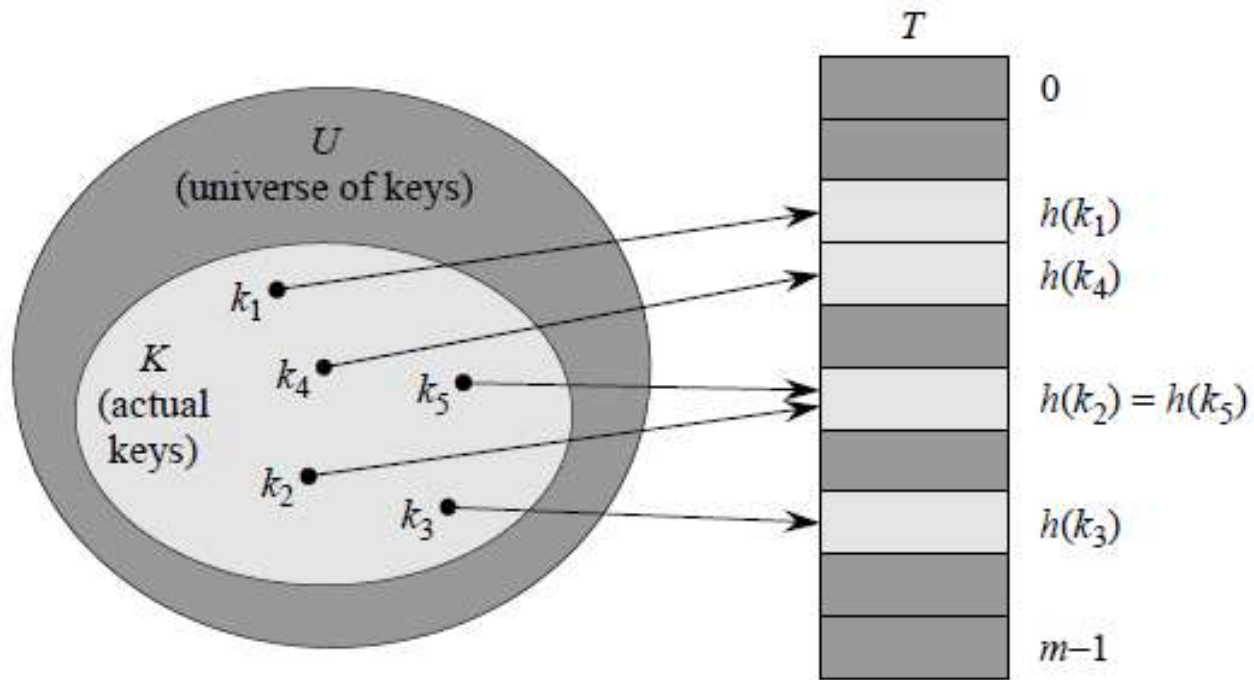
HASH TABLES

One solution to this problem involves limiting the size of our table to some value $m \ll |U|$. We then use a *hash function* $h: [0, |U|) \rightarrow [0, m)$ to determine which slot in the *hash table* the element with key k should be placed into.



HASH TABLES

Of course, the issue with this approach is that multiple possible keys can be mapped to the same slot. In the given example, k_2 and k_5 hash to the same slot – this is called a *collision*.

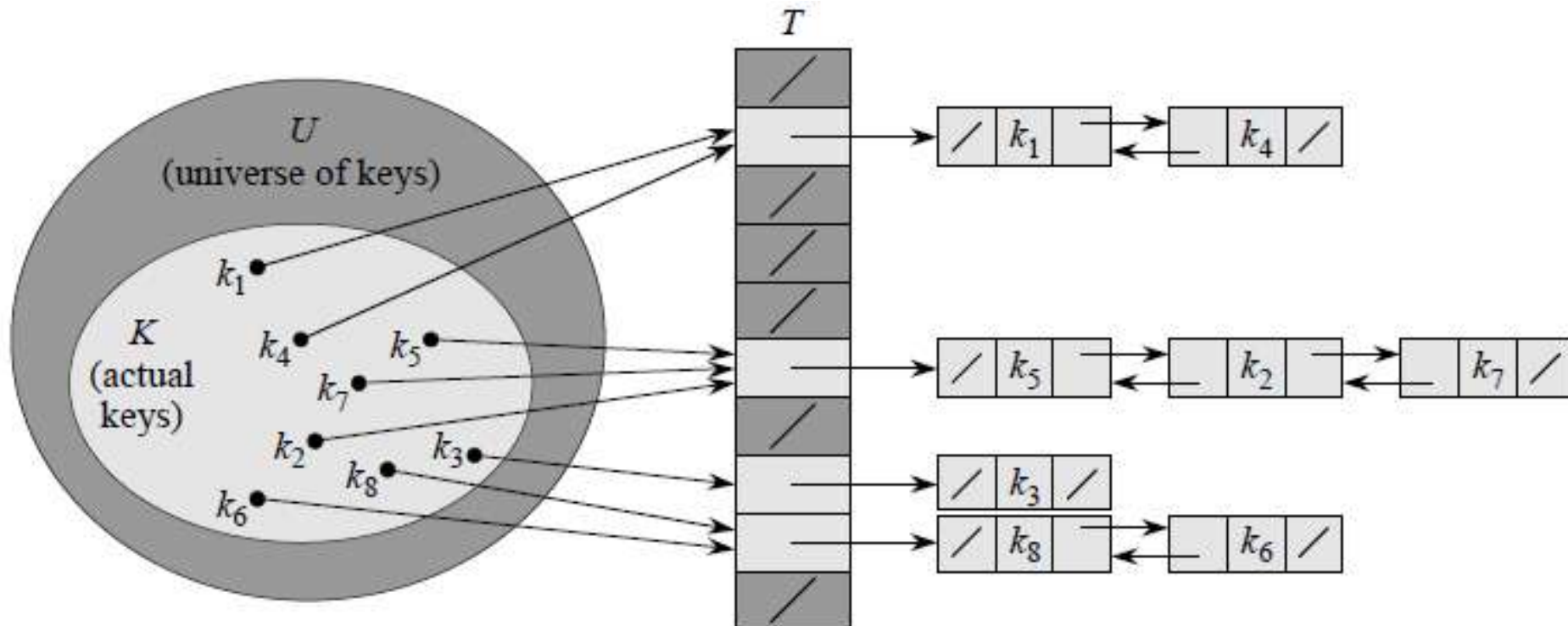


A good hash function will spread things out as best it can to avoid collisions. (This is actually where the term “hash” comes from – it jumbles things up as best it can.)

However, since $m < |U|$, the Pigeonhole Principle tells us that no hash function can be completely collision-free. Thus, we need a way to deal with them.

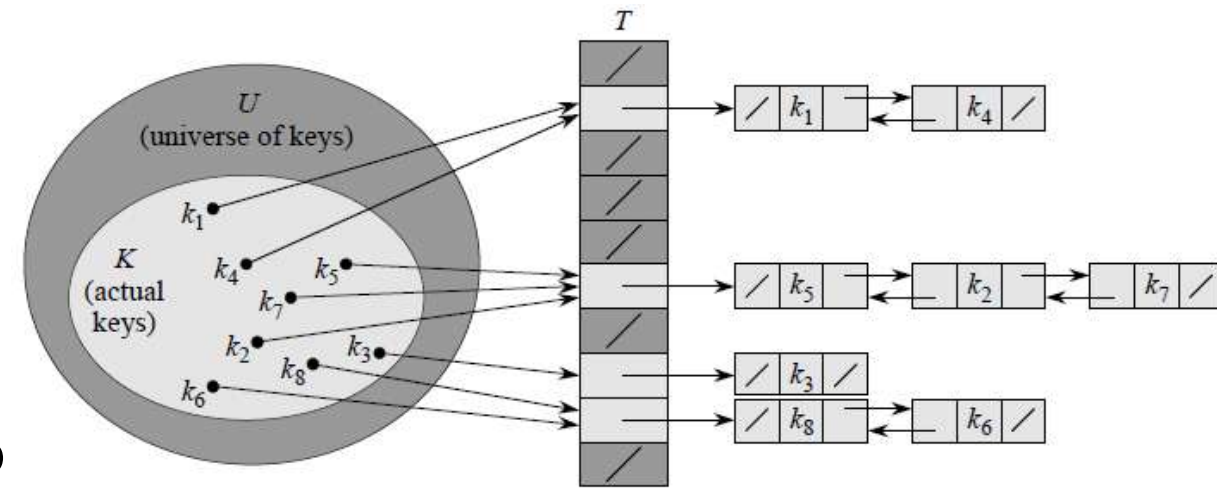
THE EASY SOLUTION: CHAINING

One obvious approach to collisions is simply to permit multiple elements per slot in the hash table. This is accomplished by *chaining* (aka *open hashing*), in which each non-empty slot in the hash table is a linked list.



ANALYSIS OF CHAINING

If we implement a hash table using chaining, then how long do our core operations take?



- Insert: Still $O(1)$ time – just insert at the head of the list.
- Delete: If we use doubly-linked lists, then if have a pointer to the object we want to delete, we can do that in $O(1)$ as well.
- Search: This, of course, is the sticking point. When the lists are short (i.e. no longer than some constant c), then it's still $O(1)$.
 - But the worst case is that every element lands in the same bucket! Then search is $O(n)$ – as bad as a linked list, with the extra overhead of the hash table.

AVERAGE CASE ANALYSIS

An algorithm with poor worst-case performance can still be useful so long as the average case is solid (cf. quicksort).

The problem is that an average case analysis is tricky because performance depends upon the specific input data and the specific hash function used.

To simplify analysis, we will (for now) assume *simple uniform hashing* – that is, the combination of input data and hash function produces an equal chance for any input key to land in any bucket, and that input items are independent of each other.

LOAD FACTOR

If our hash table has m slots and we wish to store n items in it, then we define the *load factor* $\alpha = n/m$. Note that this can be less than, equal to, or greater than 1.

If we know n ahead of time, then we can (and do) select m (subject to space limitations) to produce any desired α .

The expected length of the list under in each slot of the hash table under simple uniform hashing is just α . The average time for an unsuccessful search is thus $\Theta(1 + \alpha)$.

Analysis for a successful search is trickier, as the likelihood of looking in any particular slot is proportional to the number of elements in that slot! But it turns out that it is still $\Theta(1 + \alpha)$.

THE MATH

We are calculating here the expected number of elements examined in a successful search.

$$\begin{aligned} & \mathbb{E} \left[\frac{1}{n} \sum_{i=1}^n \left(1 + \sum_{j=i+1}^n X_{ij} \right) \right] \\ &= \frac{1}{n} \sum_{i=1}^n \left(1 + \sum_{j=i+1}^n \mathbb{E}[X_{ij}] \right) \quad (\text{by linearity of expectation}) \\ &= \frac{1}{n} \sum_{i=1}^n \left(1 + \sum_{j=i+1}^n \frac{1}{m} \right) \\ &= 1 + \frac{1}{nm} \sum_{i=1}^n (n - i) \\ &= 1 + \frac{1}{nm} \left(\sum_{i=1}^n n - \sum_{i=1}^n i \right) \\ &= 1 + \frac{1}{nm} \left(n^2 - \frac{n(n+1)}{2} \right) \quad (\text{by equation (A.1)}) \\ &= 1 + \frac{n-1}{2m} \\ &= 1 + \frac{\alpha}{2} - \frac{\alpha}{2n}. \end{aligned}$$

DEFINING A GOOD HASH FUNCTION

A good hash function has these properties:

- It must be fast to calculate. Since it is used in every insert and search operation, a slow hash function seriously degrades performance.
- It must be deterministic. (Otherwise, we can't find things again after we put them in the table.)
- It should distribute input keys as uniformly as possible.
 - Note that this is insufficient to guarantee simple uniform hashing, because input keys are often not independent.
- For many applications, we would like keys that are close (per some similarity metric) to not consistently hash to slots that are close together in the table.

THE DIVISION METHOD

Usually the first hash function taught is the modulus operator: $h(k) = k \bmod m$.

The only choice to make here is the value of m .

Observe that when m is a power of 2, then $h(k)$ is just grabbing the $\lg m$ lowest-order bits of k . Often this is bad.

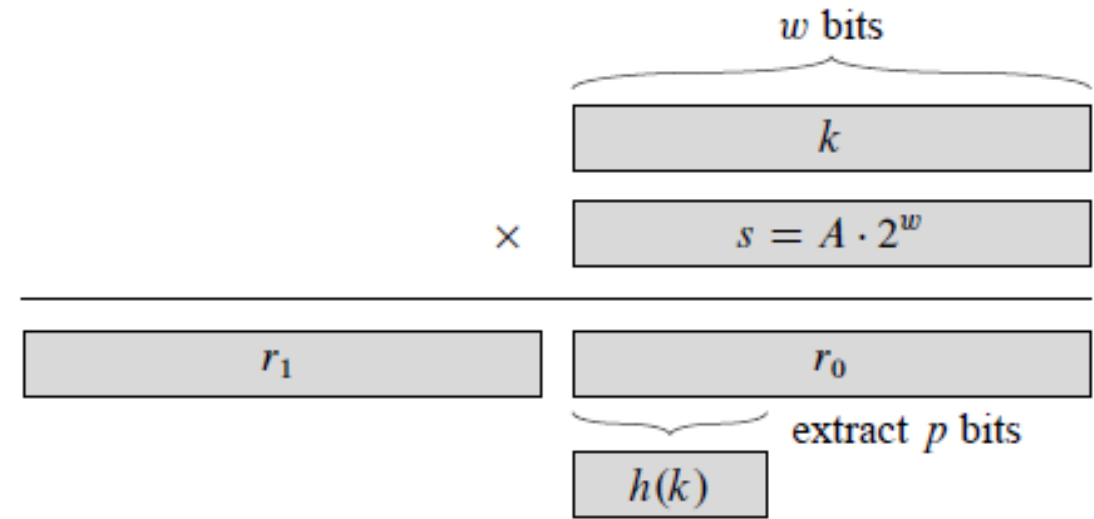
Number theory tells us that prime numbers generally make for good choices of m , so long as they are not close to powers of 2. (In particular, Mersenne primes, which equal $2^p - 1$, are poor choices.)

THE MULTIPLICATION METHOD

Instead of using the mod operator, we can instead multiply by a fixed value and then grab certain bits to be the value of $h(k)$.

A nice advantage of this is that we now can have $m = 2^p$; having m be a power of two is usually an efficient use of space, especially when the hash table is implemented in hardware.

The conceit here is that we are multiplying by some fraction A that is between 0 and 1, and then taking the top p bits of the fractional part of the answer. Some values of A work better than others; Knuth suggests $1/\phi = (\sqrt{5} - 1)/2$ as one that is likely to work well.



UNIVERSAL HASHING

Both the division method and the multiplication method see use in practice, as they meet the requirements: they're fast and (with correctly chosen parameters) distribute keys fairly uniformly.

However, a maliciously chosen set of input keys could still produce worst-case behavior. Ideally, our system would be proof against that sort of attack: No matter what set of input keys we are handed, we would like the same (low) probability of getting a very long list in any single bucket.

Our trick will be to select the hash function itself at random from a predefined collection of functions.

UNIVERSAL HASHING

Given a finite collection of hash functions $\mathcal{H}: U \rightarrow [0, m)$, we call $\mathcal{H}$ a *universal family* of hash functions if it has the property that, for every distinct $k_1, k_2 \in U$, the number of functions $h \in \mathcal{H}$ in which k_1 and k_2 collide is at most $|\mathcal{H}|/m$.

In other words, selecting an $h \in \mathcal{H}$ uniformly at random means that the probability that $h(k_1) = h(k_2)$ is at most $1/m$, which is the same as if the slots for k_1 and k_2 were chosen according to simple uniform hashing.

Theorem: If a hash function h is chosen uniformly at random from a universal family $\mathcal{H}$, then the average time needed for the search function is $O(1 + \alpha)$. (Proof omitted – it's a bunch more expected value calculations.)

UNIVERSAL FAMILY EXAMPLE

Imagine that the items we wish to hash are IPv4 addresses. These are always of the form $128.113.126.71$, where the four values are in the range $[0,255]$. Let these values be x_1, x_2, x_3, x_4 .

We will let $m = 257$: a prime number. Our family of hash functions $\mathcal{H}$ will be this set of functions:

$$h_a(x_1, x_2, x_3, x_4) = (a_1x_1 + a_2x_2 + a_3x_3 + a_4x_4) \bmod m$$

where the a_i are integers in the range $[1,256]$.

To select a specific hash function from this family, we simply choose the four coefficients a_i .

Proving this family is universal is not hard – it uses indicator random variables to check if $h(k_2) = h(k_1)$ for some given k_1 .

GENERAL UNIVERSAL HASHING

There are many universal hashing families but the most commonly used one assumes that our keys are nonnegative integers. A prime number p is chosen larger than the max key.

The family of hash functions is then

$$h_{ab}(k) = ((ak + b) \bmod p) \bmod m$$

where a, b are integers with $a \in (0, p)$ and $b \in [0, p)$.

Elementary number theory tells us the hash values are uniformly distributed whenever the coefficients are chosen uniformly at random. (Each selection of a and each selection of b is essentially a permutation of the possible outputs.)

COMPLETE THE POLL!

Make sure to complete the left-hand poll – it's the second attendance poll for the day. On the right is the standard end-of-class feedback form.

